Факультет Программной Инженерии и Компьютерной техники

«Основы профессиональной деятельности»

Лабораторная работа №5

Вариант №1231353

Выполнил: Малых Кирилл Романович

Группа: P3132

Преподаватель: Жук Иван Александрович

Санкт-Петербург, 2026 г.

Содержание

1. Порядок выполнения работы 3

4. Трассировка программы для первых 2-х символов строки 8

5. Дополнительное задание 9

6. Вывод по работе 10

7. Список использованной литературы 11

1. Порядок выполнения работы

1. Программа осуществляет асинхронный вывод данных на ВУ-1
2. Программа начинается с адреса $4FA_{16}$. Размещаемая строка находится по адресу $5AD_{16}$.
3. Строка должна быть представлена в кодировке КОИ-8.
4. Формат представления строки в памяти: АДР1: СИМВ2 СИМВ1 АДР2: СИМВ4 СИМВ3 ... СТОП_СИМВ.
5. Ввод или вывод строки должен быть завершен по символу с кодом 0A (NL). Стоп символ является обычным символом строки и подчиняется тем же правилам расположения в памяти что и другие символы строки.

2. Описание и код программы

Адрес	Код команды/данные ячейки	Мнемоника	Описание выполняемой инструкции/хранящихся данных
4F8	05AD	Current	Адрес текущей ячейки с 2-мя символами
4F9	0000	tempValue	Адрес для хранения промежуточных значений
4FA	FF	oneByteRemain	Маска для выделения младшего байта
4FB	000A	endSymbol	Стоп-символ (NL)
4FC	0200	CLA	Очистка аккумулятора: 0 → AC
4FD	AAFA	LD (IP-6)+	Загрузка из ячейки памяти в аккумулятор: AR(M(4FE-6=4F8)) → AC
4FE	EEFA	ST (IP-6)	Сохранение данных из аккумулятора в ячейку: AC → M(4FF-6=4F9)
4FF	0C00	PUSH	Запись данных из аккумулятора в стек: AC → -(SP)
500	DE08	CALL (IP+8)	Вызов подпрограммы: SP – 1 → SP, IP → (SP), (501+8=509) → IP
501	0800	POP	Чтение + Удаление элемента из стека: (SP)+ → AC
502	AEF8	LD (IP-10)	Загрузка из ячейки памяти в аккумулятор: M(503-10=4F9) → AC
503	0680	SWAB	Обмен старшего и младшего байта в аккумуляторе
504	0C00	PUSH	Запись данных из аккумулятора в стек: AC → -(SP)
505	DE03	CALL (IP+3)	Вызов подпрограммы: SP – 1 → SP, IP → (SP), (506+3=509) → IP
506	0800	POP	Чтение + Удаление элемента из стека: (SP)+ → AC
507	CEF5	JUMP (IP-11)	Выполнить безусловный переход IP = IP-11= 4FD
508	0100	HLT	Остановка работы программы
509	1203	IN 3	Ожидание готовности вывода символа на КВУ-1
50A	2F40	AND #40	Логическое “И” с прямой загрузкой операнда: AC ^ 40 → AC
50B	F0FD	BEQ (IP-3)	Переход, если равно: Если Z == 0, то IP = IP + 1 – 3 = 509
50C	AC01	LD (SP+1)	Загрузка данных в аккумулятор: M(SP+1) → AC
50D	2EEC	AND (IP-20)	Логическое “И”: AC ^ M(50E-20=4F9) → AC
50E	1302	OUT 2	Вывод данных на КВУ-1
50F	7EEB	CMP (IP-21)	Выставление признаков результатов:

			AC-M(510-21=4FB) → NZVC
510	F0F7	BEQ (511-3=508)	Переход, если равно: Если Z==0, то IP=IP+1-3=508
511	0A00	RET	Возврат из подпрограммы (SP)+ → IP
…	…	…	…
5AD	BBAA	2 и 1 символы соответственно	Младший байт хранит чётный символ Старший байт хранит нечётный символ
…	…	…	…
5AD+n (< 7F6)	ZZYY	((n+1)*2) и (n*2+1) символы соответственно	Младший байт хранит чётный символ Старший байт хранит нечётный символ

Код программы, выполненной на Ассемблере

https://github.com/tetraminomusic/ITMO_2sem/blob/main/ОПД/Lab_5/ams.bcomp

Расположение данных в памяти:

4F8-4FB – Константы и ячейки для промежуточных вычислений

4FC-508 – основная программа;

509-511 – подпрограмма printCharProgramm;

5AD-? – ячейки, имеющие по 2 символа, представленных в кодировке КОИ-8 R.

3. ОПИ и ОДЗ

ОПИ

Current – 11-ти разрядная ячейка, хранящая адрес текущей рассматриваемой ячейки;

tempValue – 16-ти разрядная ячейка, временно хранящая два символа в кодировке КОИ-8;

oneByteRemain – набор из 8-ми логических значений, является константой;

endSymbol – однобайтовый символ NL из кодировки КОИ-8, является константой;

{5AD, 5AE, …, ?} – 16-ти разрядные ячейки, хранящие в себе по два символа из кодировки КОИ-8;

ОДЗ

Пусть **char** – переменная для символа в кодировке КОИ-8 R.

Найдём ОДЗ для строки, которая будет представлять из себя диапазон допустимых для вывода в БЭВМ символов. Так как КОИ-8 R в начале таблицы имеет обычный ASCII, то сразу удалим управляющие символы с первых 32 позиций и символ DEL в конце ASCII таблицы. Оставшиеся символы из верхней части таблицы КОИ-8 являются печатными. Получившееся ОДЗ:

$$\text{char} \notin [0,31_{10}] \cup \{127_{10}\};$$

Максимальное количество символов будет являться удвоенная разность между максимально набираемой вершиной стека и ячейка, начиная с которой в памяти будут храниться символы. Учитываем, что один символ должен быть стоп-символом:

$$N_{max(\text{символы})} = (7F5_{16} - 5AD_{16} + 1) * 2 - 1 = 1179$$

Итоговое ОДЗ

$$\begin{cases} current \in [05\text{AD}; 7\text{F}5] \\ tempValue \in [0;\ 65\,535] \\ oneByteRemain = FF_{16} \\ endSymbol = 0A_{16} \\ N_{\text{символы}} \in [0;\ 1179] \\ \text{char} \in [32;\ 126] \cup [128;\ 255] \end{cases}$$

4. Трассировка программы для первых 2-х символов строки

Слово: “F00N4OZA”

КОИ-8: $(46\ 30\ 30\ 4E\ 34\ 4F\ 5A\ 41)_{16}$

UTF-8: $(46\ 30\ 30\ 4E\ 34\ 4F\ 5A\ 41)_{16}$

UTF-16: $(46\ 00\ 30\ 00\ 30\ 00\ 4E\ 00\ 34\ 00\ 4F\ 00\ 5A\ 00\ 41\ 00)_{16}$

Выполняемая команда		Содержимое регистров процессора после выполнения команды									Изменённые ячейки	
Адрес	Код	IP	CR	AR	DR	SP	BR	AC	PS	NZVC	Адрес	Новый код
4F8	current	-	-	-	-	-	-	-	-	-	-	-
4F9	tempValue	-	-	-	-	-	-	-	-	-	-	-
4FA	oneByteRemain	-	-	-	-	-	-	-	-	-	-	-
4FB	endSymbol	-	-	-	-	-	-	-	-	-	-	-
5AD	3046	-	-	-	-	-	-	-	-	-	-	-
4FC	0200	4FD	0200	4FC	0200	000	04FC	0000	004	0100	-	-
4FD	AAFA	4FE	AAFA	5AD	3046	000	FFFA	3046	000	0000	-	-
4FE	EEFA	4FF	EEFA	4F9	3046	000	FFFA	3046	000	0000	4F9	3046
4FF	0C00	500	0C00	7FF	3046	7FF	04FF	3046	000	0000	7FF	3046
500	DE08	509	DE08	7FE	0501	7FE	0509	3046	000	0000	-	-
509	1203	50A	1203	509	1203	7FE	0509	3040	000	0000	-	-
50A	2F40	50B	2F40	50A	0040	7FE	0040	0040	000	0000	-	-
50B	F0FD	50C	F0FD	50B	F0FD	7FE	050B	0040	000	0000	-	-
50C	AC01	50D	AC01	7FF	3046	7FE	0001	3046	000	0000	-	-
50D	2EEC	50E	2EEC	4FA	00FF	7FE	FFEC	0046	000	0000	-	-
50E	1302	50F	1302	50E	1302	7FE	050E	0046	000	0000	DR 2	46
50F	7EEB	510	7EEB	4FB	000A	7FE	FFEB	0046	001	0001	-	-
510	F0F7	511	F0F7	510	F0F7	7FE	0510	0046	001	0001	-	-
511	0A00	501	0A00	7FE	0501	7FF	0511	0046	001	0001	-	-
501	0800	502	0800	7FF	3046	000	0501	3046	001	0001	-	-
502	AEF6	503	AEF6	4F9	3046	000	FFF6	3046	001	0001	-	-
503	0680	504	0680	503	0680	000	0503	4630	001	0001	-	-
504	0C00	505	0C00	7FF	4630	7FF	0504	4630	001	0001	7FF	4630
505	DE03	509	DE03	7FE	0506	7FE	0509	4630	001	0001	-	-
509	1203	50A	1203	509	1203	7FE	0509	3040	000	0000	-	-
50A	2F40	50B	2F40	50A	0040	7FE	0040	0040	000	0000	-	-
50B	F0FD	50C	F0FD	50B	F0FD	7FE	050B	0040	000	0000	-	-
50C	AC01	50D	AC01	7FF	4630	7FE	0001	4630	000	0000	-	-
50D	2EEC	50E	2EEC	4FA	00FF	7FE	FFEC	0030	000	0000	-	-
50E	1302	50F	1302	50E	1302	7FE	050E	0030	000	0000	DR 2	30
50F	7EEB	510	7EEB	4FB	000A	7FE	FFEB	0030	001	0001	-	-
510	F0F7	511	F0F7	510	F0F7	7FE	0510	0030	001	0001	-	-
511	0A00	501	0A00	7FE	0501	7FF	0511	0030	001	0001	-	-

5. Дополнительное задание

Реализовать программу, которая способна выводить ASCII графику в текстовый принтер БЭВМ:

Реализованная программа на Ассемблере:

https://github.com/tetraminomusic/ITMO_2sem/blob/main/ОПД/Lab_5/polyakov.asm

6. Вывод по работе

В ходе данной лабораторной работы я познакомился с ассемблером в БЭВМ и смог с помощью него разработать программу более удобным и приятным способом, чем я делал до этого. Также я окунулся в тему с операциями ввода-вывода внутри БЭВМ, узнал, как правильно корректно обрабатывать взаимодействие с внешними устройствами. Хотелось бы и дальше писать программы для БЭВМ исключительно на ассемблере, чтобы не ломать себе пальцы.

7. Список использованной литературы

1) Кириллов В.В. Архитектура **базовой ЭВМ** – СПб: СПбГУ ИТМО, 2010. – 144 с.

2) Клименков С.В. – “ОПД 2020_21, #1-4. Архитектура и система команд БЭВМ”.

URL: https://vkvideo.ru/video-34559124_456239019

3) Клименков С.В. – “Как построить ферму мобов в майнкрафте”

URL: https://vkvideo.ru/video-34559124_456239020